

Practical(Q2)

Name: Anthony Nissy

Rollno: 2520030064

Section: 8

Title: File Copying Using Linux System Calls and System Call Tracing Using strace

Aim: To develop a C program that uses the Linux system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Also, to use the `strace` utility to trace the system calls generated by the command:

`cat sample.txt`

and analyse the sequence of system calls and identify the kernel services involved.

Objective:

- To understand file handling using Linux system calls.
- To learn how `open()` opens a file.
- To learn how `read()` reads data from a file.
- To learn how `write()` writes data to another file.
- To learn how `close()` closes an opened file.
- To understand the transition between **user space and kernel space**.
- To use `strace` for tracing system calls.
- To identify the kernel services involved in executing `cat sample.txt`.

Theory:

Linux provides system calls that allow user programs to request services from the kernel.

For file operations, the important system calls are:

1. `open()`

The `open()` system call opens a file and returns a **file descriptor**.

Example:

```
int fd = open("source.txt", O_RDONLY);
```

Here, `O_RDONLY` opens the file in read-only mode.

2. `read()`

The `read()` system call reads data from an opened file into a buffer.

```
read(fd, buffer, sizeof(buffer));
```

It returns the number of bytes actually read.

3. write()

The write() system call writes data from a buffer to a file.

write(fd, buffer, bytes);

4. close()

The close() system call closes the file descriptor after the file operation is completed.

close(fd);

User Space and Kernel Space

Linux divides execution into two important areas:

User Space

This is where normal applications such as our C program and the cat command execute.

Kernel Space

This is where the Linux kernel manages:

- Files
- Memory
- Processes
- Hardware
- Input/output operations
- File systems

When a C program calls open(), read(), write(), or close(), control transitions from **user space to kernel space**.

Control Flow

User Program

|

| open()

↓

Kernel

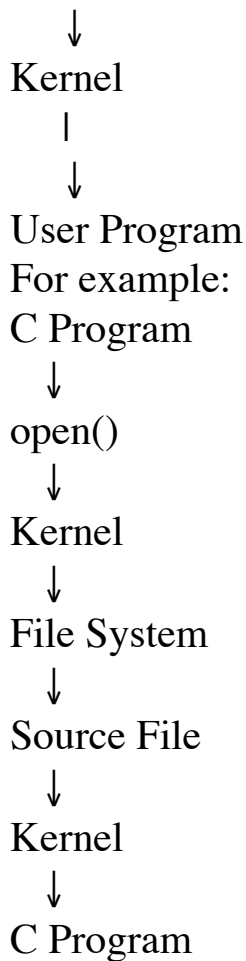
|

| File System

↓

Disk / Storage

|



The same process occurs for read(), write(), and close().

Algorithm:

- Start the program.
- Open the source file using open() in read-only mode.
- Open/create the destination file using open().
- Read data from the source file using read().
- Write the data to the destination file using write().
- Repeat the read and write operations until the complete file is copied.
- Close the source file using close().
- Close the destination file using close().
- Display a success message.
- Stop.

System Calls Used:

System Call	Purpose
-------------	---------

open()	Opens or creates a file
read()	Reads data from a file
write()	Writes data to a file
close()	Closes a file descriptor

C Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead;
    // Open source file for reading
    source = open("source.txt", O_RDONLY);
    if (source == -1)
    {
        perror("Error opening source file");
        return 1;
    }
    // Open destination file for writing
    // Create it if it does not exist
    // Give read and write permissions to the owner
    destination = open("destination.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);
    if (destination == -1)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }
    // Read and write until the complete file is copied
    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
```

```

    if (write(destination, buffer, bytesRead) != bytesRead)
    {
        perror("Error writing to destination file");
        close(source);
        close(destination);
        return 1;
    }
}if (bytesRead == -1)
{
    perror("Error reading source file");
}
else{
    printf("File copied successfully.\n");
}
// Close both files
close(source);
close(destination);
return 0;
}

```

OUTPUT:

Part 2: strace System Call Tracing

Aim: To trace all system calls generated by the command:

cat sample.txt

using the strace utility and analyze the kernel services involved.

What is strace?

strace is a Linux debugging and diagnostic utility used to monitor the **system calls made by a process**.

For example:

strace cat sample.txt

This shows the system calls made by the cat command.

Important System Calls in cat sample.txt

When you execute:

cat sample.txt

the exact output varies depending on your Ubuntu version, libraries, architecture, and environment. However, the important file-related sequence is typically similar to:

```
execve("../cat", ...)
openat(..., "sample.txt", O_RDONLY)
read(..., ...)
write(1, ...)
read(..., ...)
close(...)
exit_group(0)
```

You may also see many additional calls related to program startup and shared libraries.

Sequence of Important System Calls:

1. execve()

```
execve(...)
```

The shell starts the cat program using execve().

Kernel service: Process/program execution.

2. openat()

```
openat(..., "sample.txt", O_RDONLY)
```

cat asks the kernel to open sample.txt.

The kernel checks the file and returns a file descriptor.

Kernel service: File-system access.

On modern Linux systems, cat commonly uses openat() rather than the older open() system call.

3. read()

```
read(fd, "Hello Linux!\n", ...)
```

The cat program requests the kernel to read the contents of sample.txt.

Kernel service: File input / storage access.

4. write()

```
write(1, "Hello Linux!\n", ...)
```

File descriptor 1 represents **standard output**, normally the terminal.

The kernel writes the contents to the terminal.

Kernel service: Output/I/O.

5. close()

close(fd)

After reading the file, cat closes the file descriptor.

Kernel service: File descriptor/resource management.

6. exit_group()

exit_group(0)

The process terminates successfully.

Kernel service: Process termination.

Simplified strace Flow:

Shell

|

| execve()

↓

cat program

|

| openat()

↓

Kernel

|

| Opens sample.txt

↓

File System

|

| read()

↓

Kernel

|

| write()

↓

Terminal

|

| close()

↓

Kernel

|
| exit_group()
↓
Process terminates

Observation:

- The C program successfully copied the contents of source.txt to destination.txt.
- open() was used to open the source and destination files.
- read() was used to read data from the source file.
- write() was used to write data to the destination file.
- close() was used to release the file descriptors.
- The strace utility successfully displayed the system calls generated by cat sample.txt.
- The cat command uses kernel services for opening, reading, writing, and closing files.
- System calls provide the interface between **user-space applications and the Linux kernel**.

Result:

The C program was successfully implemented using the open(), read(), write(), and close() system calls to copy a file.

The strace utility was also successfully used to trace the system calls generated by cat sample.txt. The experiment demonstrated how user-space programs request file and process-related services from the Linux kernel through system calls.